# 05_fully_connected_nets_repo

July 8, 2026

## 5. Multi-Layer Fully Connected Networks on CIFAR-10

This notebook adapts the original CS231n `FullyConnectedNets.ipynb` exercise to the **local repo-friendly workflow** used by the other notebooks in `demos/`.

We keep the assignment's experimentation flow, but make it work directly with this repository: - run locally from `demos/` - import reusable implementations from `src/` - load CIFAR-10 from the repo's `data/` directory - preserve the original gradient checks, optimizer experiments, and model-tuning tasks

### 5.1 Related source files

- `src/models/fc_net.py`
- `src/layers.py`
- `src/optim.py`
- `src/utils/data.py`
- `src/utils/gradient_check.py`
- `src/utils/solver.py`

### 5.2 Why deeper fully connected networks?

A two-layer network is already expressive enough to learn nonlinear decision boundaries, but deeper fully connected networks let us compose several affine + nonlinearity blocks into a richer function class.

That extra depth introduces two practical challenges that this notebook focuses on: - **implementation discipline**: every layer must expose a clean forward and backward interface; - **optimization stability**: deeper networks are more sensitive to initialization and update rules.

The assignment is structured around those two ideas: first make the modular implementation correct, then make optimization work well enough to train a useful classifier on CIFAR-10.

### 5.3 Setup

The setup below mirrors the earlier repo notebooks: - add the repository root to `sys.path`, - import the local model, solver, and gradient-check helpers from `src/`, - create the `artifacts/` directory for saved models, - and configure plotting defaults for notebook work.

[1]:
```
from __future__ import print_function
```

```
import sys
import tarfile
import urllib.request
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

repo_root = Path().resolve().parent
if str(repo_root) not in sys.path:
    sys.path.append(str(repo_root))

from src.models.fc_net import FullyConnectedNet
from src.utils.data import load_CIFAR10
from src.utils.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from src.utils.solver import Solver

(repo_root / 'artifacts').mkdir(parents=True, exist_ok=True)

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

%load_ext autoreload
%autoreload 2


def rel_error(x, y):
    """Return the maximum relative error."""
    return np.max(np.abs(x - y) / np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

### 5.4 Load and preprocess CIFAR-10

The original notebook used `get_CIFAR10_data()` from the course codebase. Here we reproduce the same preprocessing locally:

1. load raw CIFAR-10 from `data/`,
2. download it automatically if it is missing,
3. split into training / validation / test subsets,
4. subtract the mean training image,
5. transpose images to channel-first format `(N, 3, 32, 32)`.

This keeps the later exercise cells close to the original notebook while matching the structure of this repo.

[2]:
```
cifar10_dir = repo_root / 'data' / 'cifar-10-batches-py'

if not cifar10_dir.exists():
    print('CIFAR-10 dataset not found. Downloading...')

    data_dir = repo_root / 'data'
    data_dir.mkdir(parents=True, exist_ok=True)

    url = 'https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz'
    archive_path = data_dir / 'cifar-10-python.tar.gz'

    urllib.request.urlretrieve(url, archive_path)

    print('Extracting dataset...')
    with tarfile.open(archive_path, 'r:gz') as tar:
        tar.extractall(path=data_dir)

    print('Download complete.')


def get_CIFAR10_data_local(
    num_training=49000,
    num_validation=1000,
    num_test=1000,
    subtract_mean=True,
):
    """Match the original assignment preprocessing using the local repo layout.
    """
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]

    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]

    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    if subtract_mean:
        mean_image = np.mean(X_train, axis=0)
        X_train = X_train - mean_image
        X_val = X_val - mean_image
        X_test = X_test - mean_image
```

```
    X_train = X_train.transpose(0, 3, 1, 2).copy()
    X_val = X_val.transpose(0, 3, 1, 2).copy()
    X_test = X_test.transpose(0, 3, 1, 2).copy()

    return {
        'X_train': X_train,
        'y_train': y_train,
        'X_val': X_val,
        'y_val': y_val,
        'X_test': X_test,
        'y_test': y_test,
    }


data = get_CIFAR10_data_local()
for k, v in list(data.items()):
    print(f'{k}: {v.shape}')
```

```
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/utils/data.py:16:
VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy
boolean but got `align=0`. Did you mean to pass a tuple to create a subarray
type? (Deprecated NumPy 2.4)
  return pickle.load(f, encoding="latin1")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

### 5.5 Initial loss and gradient check

Start by reading through `FullyConnectedNet` in `src/models/fc_net.py`.

At this stage, the core task is to implement: - parameter initialization, - the forward pass through an arbitrary number of hidden layers, - and the backward pass that produces gradients for all parameters.

The numeric gradient check below is the quickest sanity test for whether those pieces are wired correctly. As in the original assignment, relative errors around `1e-7` or smaller are a good sign.

[3]:
```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
```

```
    print('Running check with reg = ', reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64,
    )

    loss, grads = model.loss(X, y)
    print('Initial loss: ', loss)

    # Most errors should be around e-7 or smaller.
    # It is acceptable to see W2 near e-5 when reg = 0.
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],␣
↪verbose=False, h=1e-5)
        print(f'{name} relative error: {rel_error(grad_num, grads[name])}')
```

```
Running check with reg =  0
Initial loss:  2.300479089768492
W1 relative error: 1.4839895515510274e-07
W2 relative error: 2.2120479308354723e-05
W3 relative error: 4.562327799137699e-07
b1 relative error: 4.660094372886962e-09
b2 relative error: 2.085654124402131e-09
b3 relative error: 1.689724888469736e-10
Running check with reg =  3.14
Initial loss:  7.052114776533016
W1 relative error: 3.904542008453064e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 3.483989217647104e-08
b1 relative error: 1.4752429377838921e-08
b2 relative error: 1.4615869332918208e-09
b3 relative error: 1.3200479211447775e-10
```

### 5.6 Can the model overfit a tiny dataset?

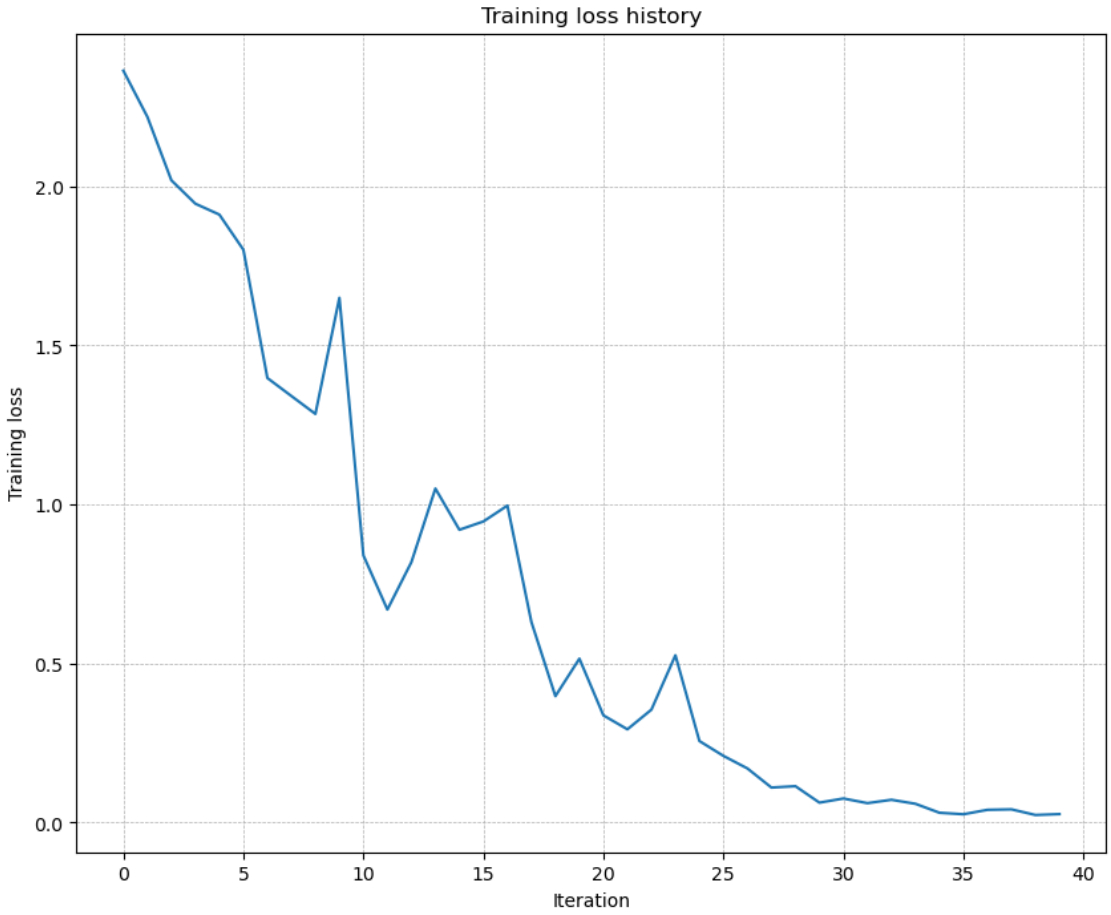
A useful debugging trick is to deliberately overfit a very small training set. If optimization and backpropagation are implemented correctly, a sufficiently expressive network should drive the training accuracy to 100% on 50 examples.

We first try a three-layer network, then a deeper five-layer one. The deeper model is usually more sensitive to learning rate and initialization scale, so this is also a practical test of optimization stability.

[4]:
```
# TODO: Use a three-layer net to overfit 50 training examples by
# tweaking the learning rate and initialization scale.

num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

weight_scale = 1e-2   # Experiment with this.
learning_rate = 1e-2  # Experiment with this.

model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64,
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 2.363364
(Epoch 0 / 20) train acc: 0.180000; val_acc: 0.108000
(Epoch 1 / 20) train acc: 0.320000; val_acc: 0.127000
(Epoch 2 / 20) train acc: 0.440000; val_acc: 0.172000
(Epoch 3 / 20) train acc: 0.500000; val_acc: 0.184000
(Epoch 4 / 20) train acc: 0.540000; val_acc: 0.181000
(Epoch 5 / 20) train acc: 0.740000; val_acc: 0.190000
(Iteration 11 / 40) loss: 0.839976
(Epoch 6 / 20) train acc: 0.740000; val_acc: 0.187000
(Epoch 7 / 20) train acc: 0.740000; val_acc: 0.183000
(Epoch 8 / 20) train acc: 0.820000; val_acc: 0.177000
```

```
(Epoch 9 / 20) train acc: 0.860000; val_acc: 0.200000
(Epoch 10 / 20) train acc: 0.920000; val_acc: 0.191000
(Iteration 21 / 40) loss: 0.337174
(Epoch 11 / 20) train acc: 0.960000; val_acc: 0.189000
(Epoch 12 / 20) train acc: 0.940000; val_acc: 0.180000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.199000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.199000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.195000
(Iteration 31 / 40) loss: 0.075911
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.182000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.201000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.207000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.185000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.192000
```




Now try a five-layer network with 100 hidden units in each hidden layer. You should still be able to overfit 50 examples within 20 epochs, but the tuning is usually less forgiving.

[5]:
```
num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

learning_rates = [1e-2, 5e-2, 1e-1, 2e-1]
weight_scales = [1e-3, 5e-3, 1e-2, 5e-2]

results = []
best_solver = None
best_model = None
best_train_acc = -1
best_lr = None
best_ws = None

for lr in learning_rates:
    for ws in weight_scales:
        model = FullyConnectedNet(
            [100, 100, 100, 100],
            weight_scale=ws,
            dtype=np.float64,
        )
        solver = Solver(
            model,
            small_data,
            print_every=10,
            num_epochs=20,
            batch_size=25,
            update_rule='sgd',
            optim_config={'learning_rate': lr},
            verbose=False,
        )
        solver.train()

        train_acc = solver.train_acc_history[-1]
        val_acc = solver.val_acc_history[-1]
        results.append((lr, ws, train_acc, val_acc))

        print(
            f"lr={lr:.1e}, ws={ws:.1e}, "
            f"train_acc={train_acc:.3f}, val_acc={val_acc:.3f}")
        if train_acc > best_train_acc:
            best_train_acc = train_acc
            best_model = model
```

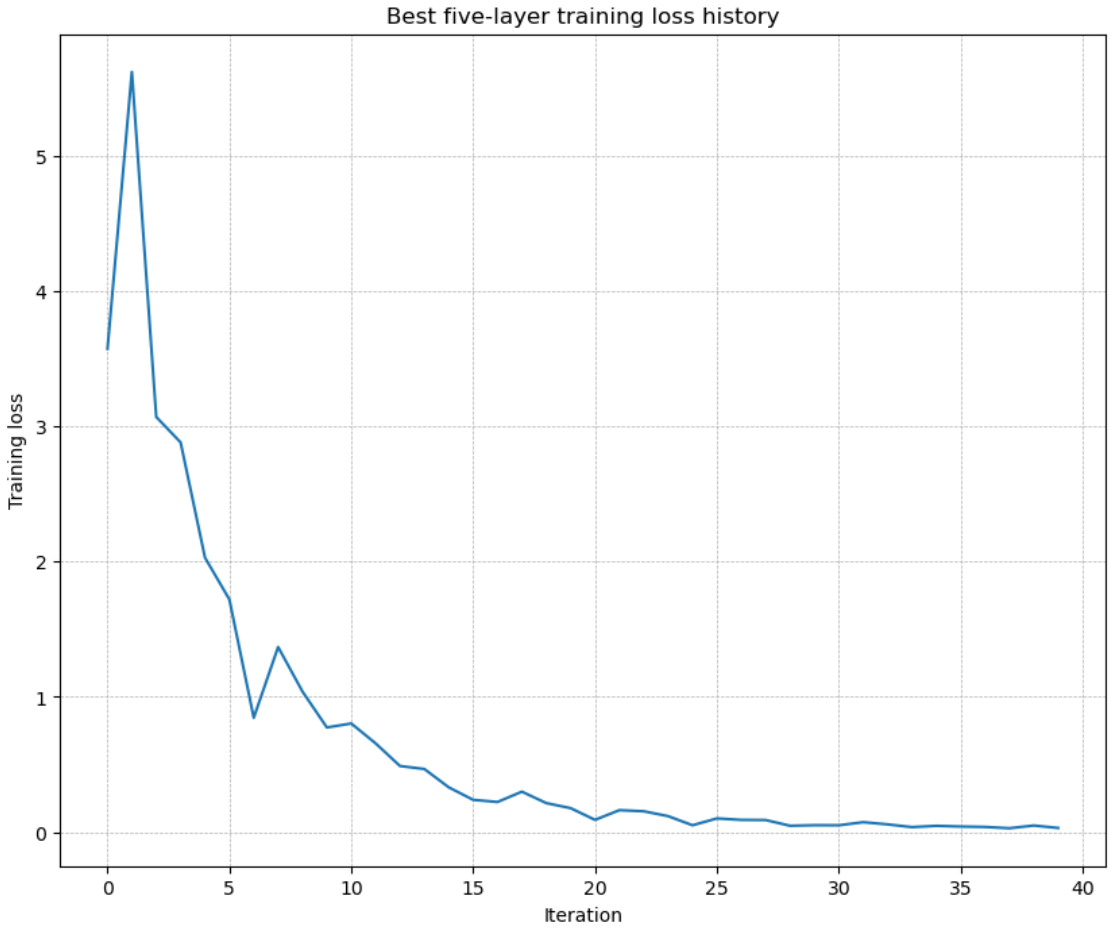
```
            best_solver = solver
            best_lr = lr
            best_ws = ws
            best_model = model
            best_solver = solver

print(f"\nBest train acc: {best_train_acc:.3f} (lr={best_lr:.1e}, ws={best_ws:.
  ↪1e})")


plt.plot(best_solver.loss_history)
plt.title('Best five-layer training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
lr=1.0e-02, ws=1.0e-03, train_acc=0.160, val_acc=0.079
lr=1.0e-02, ws=5.0e-03, train_acc=0.160, val_acc=0.112
lr=1.0e-02, ws=1.0e-02, train_acc=0.160, val_acc=0.112
lr=1.0e-02, ws=5.0e-02, train_acc=1.000, val_acc=0.150
lr=5.0e-02, ws=1.0e-03, train_acc=0.160, val_acc=0.112
lr=5.0e-02, ws=5.0e-03, train_acc=0.160, val_acc=0.079
lr=5.0e-02, ws=1.0e-02, train_acc=0.180, val_acc=0.112
```

```
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/layers.py:886: RuntimeWarning:
divide by zero encountered in log
  loss = -np.sum(np.log(probs[np.arange(N), y])) / N
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/layers.py:30: RuntimeWarning:
overflow encountered in dot
  out = x.reshape(x.shape[0], -1).dot(w) + b
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/models/fc_net.py:348:
RuntimeWarning: overflow encountered in square
  loss += 0.5 * self.reg * np.sum(self.params["W%d" % i] ** 2)
/Users/cao20/opt/anaconda3/envs/cs231n/lib/python3.11/site-
packages/numpy/_core/fromnumeric.py:83: RuntimeWarning: overflow encountered in
reduce
  return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/models/fc_net.py:348:
RuntimeWarning: invalid value encountered in scalar multiply
  loss += 0.5 * self.reg * np.sum(self.params["W%d" % i] ** 2)
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/layers.py:30: RuntimeWarning:
invalid value encountered in dot
  out = x.reshape(x.shape[0], -1).dot(w) + b
```

```
lr=5.0e-02, ws=5.0e-02, train_acc=0.080, val_acc=0.087
lr=1.0e-01, ws=1.0e-03, train_acc=0.160, val_acc=0.112
lr=1.0e-01, ws=5.0e-03, train_acc=0.160, val_acc=0.079
lr=1.0e-01, ws=1.0e-02, train_acc=0.220, val_acc=0.096
```

```
lr=1.0e-01, ws=5.0e-02, train_acc=0.080, val_acc=0.087
lr=2.0e-01, ws=1.0e-03, train_acc=0.160, val_acc=0.079
lr=2.0e-01, ws=5.0e-03, train_acc=0.160, val_acc=0.112
lr=2.0e-01, ws=1.0e-02, train_acc=0.200, val_acc=0.105
lr=2.0e-01, ws=5.0e-02, train_acc=0.080, val_acc=0.087

Best train acc: 1.000 (lr=1.0e-02, ws=5.0e-02)
```

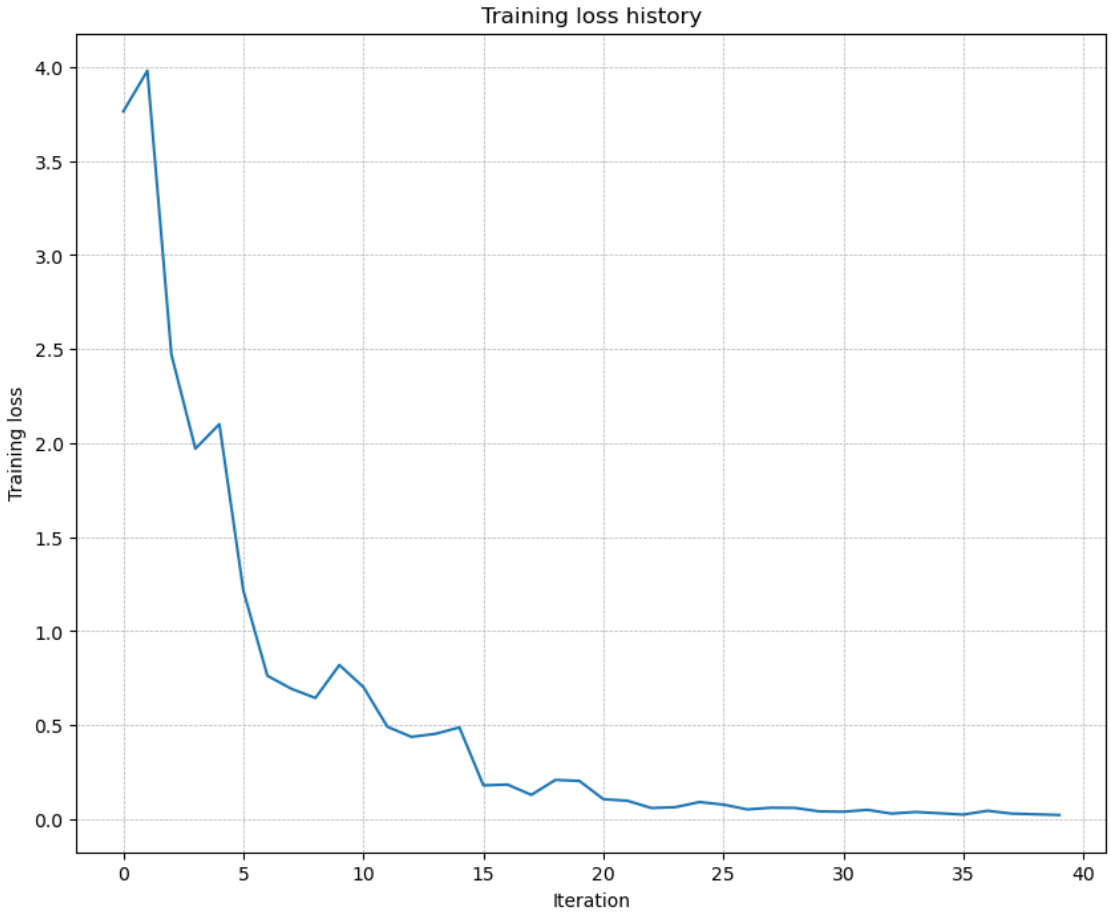



[6]:
```python
# TODO: Use a five-layer net to overfit 50 training examples by
# tweaking the learning rate and initialization scale.

num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}
```

```
learning_rate = 1e-2  # Experiment with this.
weight_scale = 5e-2   # Experiment with this.

model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64,
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 3.764345
(Epoch 0 / 20) train acc: 0.300000; val_acc: 0.131000
(Epoch 1 / 20) train acc: 0.300000; val_acc: 0.110000
(Epoch 2 / 20) train acc: 0.420000; val_acc: 0.107000
(Epoch 3 / 20) train acc: 0.680000; val_acc: 0.139000
(Epoch 4 / 20) train acc: 0.800000; val_acc: 0.143000
(Epoch 5 / 20) train acc: 0.920000; val_acc: 0.166000
(Iteration 11 / 40) loss: 0.703532
(Epoch 6 / 20) train acc: 0.920000; val_acc: 0.144000
(Epoch 7 / 20) train acc: 0.980000; val_acc: 0.150000
(Epoch 8 / 20) train acc: 1.000000; val_acc: 0.140000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.149000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.160000
(Iteration 21 / 40) loss: 0.106949
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.149000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.158000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.151000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.157000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.161000
(Iteration 31 / 40) loss: 0.040295
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.165000
```

```
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.167000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.165000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.166000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.163000
```




**Inline Question 1**

Did you notice anything about the comparative difficulty of training the three-layer network versus the five-layer network? Which network is more sensitive to the initialization scale, and why?

**Answer.**

The five-layer network is usually more sensitive. With more stacked affine and ReLU blocks, small changes in the initialization scale get amplified across depth: activations can shrink toward zero, blow up, or leave too many ReLUs inactive. That makes the deeper network harder to optimize with plain SGD, while the shallower three-layer network tends to tolerate a wider range of scales.

### 5.7 Update rules

So far we have used plain stochastic gradient descent. For deeper networks, optimization often improves significantly once we add momentum or adaptive per-parameter step sizes.

In this repo those update rules live in `src/optim.py`. The next sections mirror the original assignment: first verify each optimizer numerically, then compare training behavior on the same deep network.

### 5.8 SGD + Momentum

Implement `sgd_momentum` in `src/optim.py`, then run the reference check below. You should see errors around `1e-8` or smaller.

[7]:
```
from src.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N * D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N * D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N * D).reshape(N, D)

config = {'learning_rate': 1e-3, 'velocity': v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
    [0.1406, 0.20738947, 0.27417895, 0.34096842, 0.40775789],
    [0.47454737, 0.54133684, 0.60812632, 0.67491579, 0.74170526],
    [0.80849474, 0.87528421, 0.94207368, 1.00886316, 1.07565263],
    [1.14244211, 1.20923158, 1.27602105, 1.34281053, 1.4096],
])
expected_velocity = np.asarray([
    [0.5406, 0.55475789, 0.56891579, 0.58307368, 0.59723158],
    [0.61138947, 0.62554737, 0.63970526, 0.65386316, 0.66802105],
    [0.68217895, 0.69633684, 0.71049474, 0.72465263, 0.73881053],
    [0.75296842, 0.76712632, 0.78128421, 0.79544211, 0.8096],
])

print('next_w error: ', rel_error(next_w, expected_next_w))
print('velocity error: ', rel_error(expected_velocity, config['velocity']))
```

```
next_w error:  8.882347033505819e-09
velocity error:  4.269287743278663e-09
```

Once momentum is implemented, compare it against vanilla SGD on a six-layer network. The loss should usually fall faster with momentum.

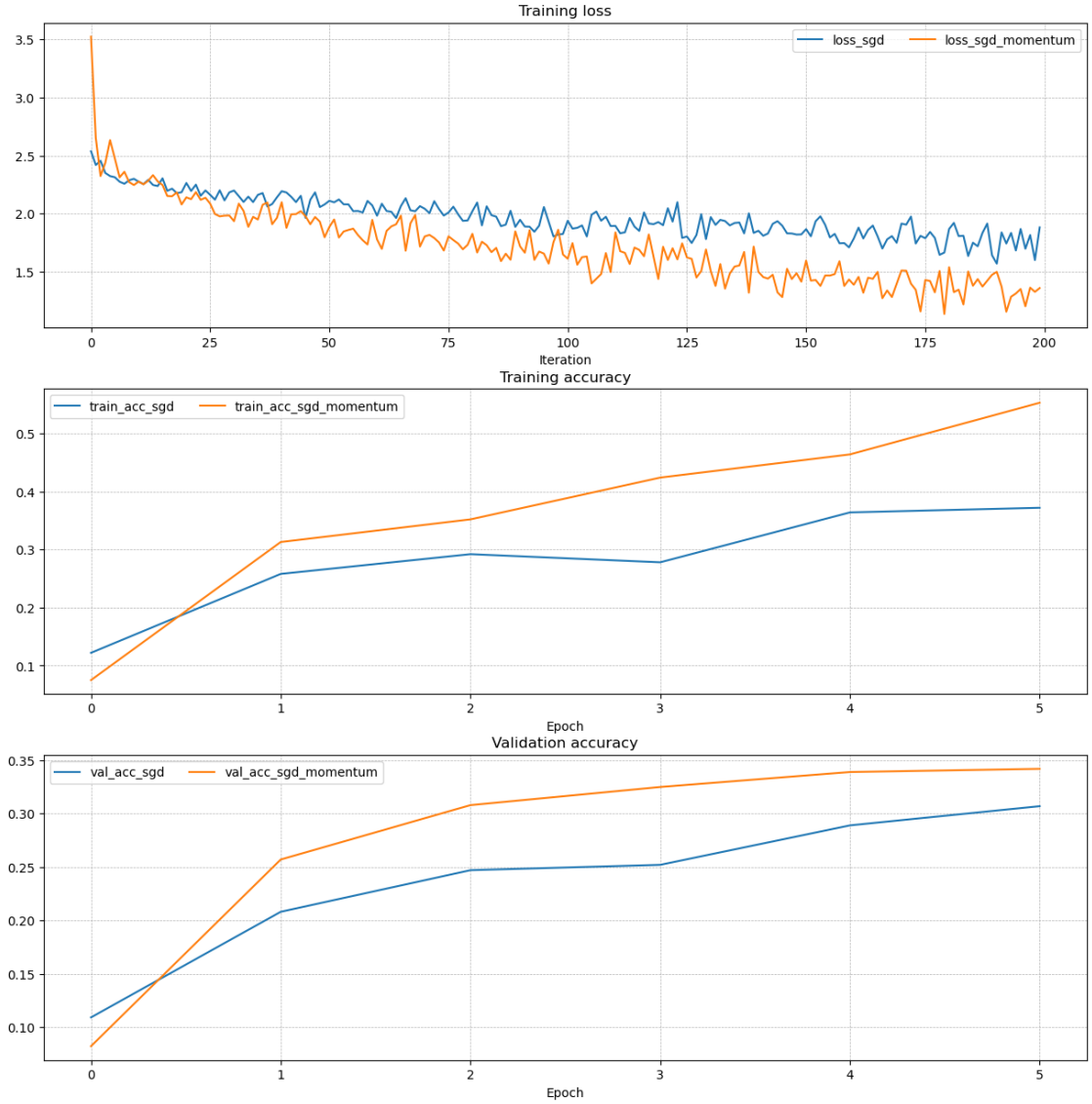
[8]:
```
num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
```

```
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2,
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f'loss_{update_rule}')
    axes[1].plot(solver.train_acc_history, label=f'train_acc_{update_rule}')
    axes[2].plot(solver.val_acc_history, label=f'val_acc_{update_rule}')

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.537440
(Epoch 0 / 5) train acc: 0.122000; val_acc: 0.109000
```

```
(Iteration 11 / 200) loss: 2.275043
(Iteration 21 / 200) loss: 2.264691
(Iteration 31 / 200) loss: 2.199818
(Epoch 1 / 5) train acc: 0.258000; val_acc: 0.208000
(Iteration 41 / 200) loss: 2.194078
(Iteration 51 / 200) loss: 2.111654
(Iteration 61 / 200) loss: 1.983303
(Iteration 71 / 200) loss: 2.043094
(Epoch 2 / 5) train acc: 0.292000; val_acc: 0.247000
(Iteration 81 / 200) loss: 2.021876
(Iteration 91 / 200) loss: 1.947910
(Iteration 101 / 200) loss: 1.940256
(Iteration 111 / 200) loss: 1.895828
(Epoch 3 / 5) train acc: 0.278000; val_acc: 0.252000
(Iteration 121 / 200) loss: 1.901793
(Iteration 131 / 200) loss: 1.971558
(Iteration 141 / 200) loss: 1.853989
(Iteration 151 / 200) loss: 1.868130
(Epoch 4 / 5) train acc: 0.364000; val_acc: 0.289000
(Iteration 161 / 200) loss: 1.786761
(Iteration 171 / 200) loss: 1.915595
(Iteration 181 / 200) loss: 1.869732
(Iteration 191 / 200) loss: 1.571201
(Epoch 5 / 5) train acc: 0.372000; val_acc: 0.307000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 3.522364
(Epoch 0 / 5) train acc: 0.075000; val_acc: 0.082000
(Iteration 11 / 200) loss: 2.277268
(Iteration 21 / 200) loss: 2.141035
(Iteration 31 / 200) loss: 1.936170
(Epoch 1 / 5) train acc: 0.313000; val_acc: 0.257000
(Iteration 41 / 200) loss: 2.099330
(Iteration 51 / 200) loss: 1.886719
(Iteration 61 / 200) loss: 1.779531
(Iteration 71 / 200) loss: 1.805186
(Epoch 2 / 5) train acc: 0.352000; val_acc: 0.308000
(Iteration 81 / 200) loss: 1.827749
(Iteration 91 / 200) loss: 1.719927
(Iteration 101 / 200) loss: 1.613538
(Iteration 111 / 200) loss: 1.840273
(Epoch 3 / 5) train acc: 0.424000; val_acc: 0.325000
(Iteration 121 / 200) loss: 1.716426
(Iteration 131 / 200) loss: 1.508264
(Iteration 141 / 200) loss: 1.500746
(Iteration 151 / 200) loss: 1.597695
(Epoch 4 / 5) train acc: 0.464000; val_acc: 0.339000
(Iteration 161 / 200) loss: 1.389639
(Iteration 171 / 200) loss: 1.512256
```

```
(Iteration 181 / 200) loss: 1.540807
(Iteration 191 / 200) loss: 1.500855
(Epoch 5 / 5) train acc: 0.553000; val_acc: 0.342000
```




## 5.9 RMSProp and Adam

Implement `rmsprop` and `adam` in `src/optim.py`, then verify them with the reference checks below. As in the original CS231n notebook, use the full Adam update with bias correction.

[9]:
```
from src.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N * D).reshape(N, D)
```

```
dw = np.linspace(-0.6, 0.4, num=N * D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N * D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737, -0.08078555, -0.02881884, 0.02316247, 0.07515774],
    [0.12716641, 0.17918792, 0.23122175, 0.28326742, 0.33532447],
    [0.38739248, 0.43947102, 0.49155973, 0.54365823, 0.59576619],
])
expected_cache = np.asarray([
    [0.5976, 0.6126277, 0.6277108, 0.64284931, 0.65804321],
    [0.67329252, 0.68859723, 0.70395734, 0.71937285, 0.73484377],
    [0.75037008, 0.7659518, 0.78158892, 0.79728144, 0.81302936],
    [0.82883269, 0.84469141, 0.86060554, 0.87657507, 0.8926],
])

print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

[10]:
```
from src.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N * D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N * D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N * D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N * D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274, -0.08544591, -0.03286534, 0.01971428, 0.0722929],
    [0.1248705, 0.17744702, 0.23002243, 0.28259667, 0.33516969],
    [0.38774145, 0.44031188, 0.49288093, 0.54544852, 0.59801459],
])
expected_v = np.asarray([
    [0.69966, 0.68908382, 0.67851319, 0.66794809, 0.65738853],
    [0.64683452, 0.63628604, 0.6257431, 0.61520571, 0.60467385],
    [0.59414753, 0.58362676, 0.57311152, 0.56260183, 0.55209767],
    [0.54159906, 0.53110598, 0.52061845, 0.51013645, 0.49966],
```

```
])
expected_m = np.asarray([
    [0.48, 0.49947368, 0.51894737, 0.53842105, 0.55789474],
    [0.57736842, 0.59684211, 0.61631579, 0.63578947, 0.65526316],
    [0.67473684, 0.69421053, 0.71368421, 0.73315789, 0.75263158],
    [0.77210526, 0.79157895, 0.81105263, 0.83052632, 0.85],
])

print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
```

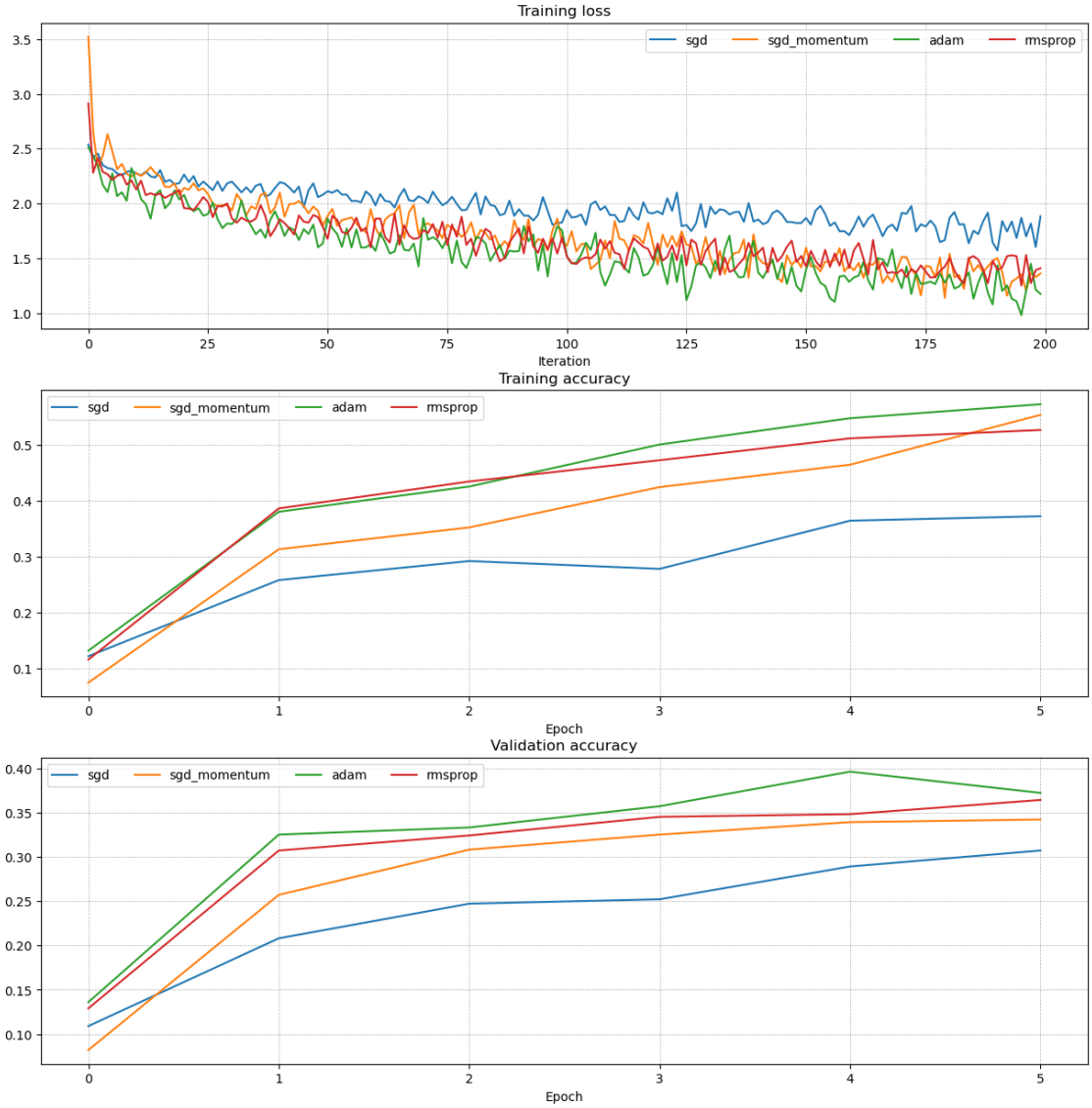
Once those optimizers are working, compare deep-network training with Adam and RMSProp. Adaptive methods typically converge faster than plain SGD on this task.

[11]:
```
learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2,
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')
```

```python
for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=update_rule)
    axes[1].plot(solver.train_acc_history, label=update_rule)
    axes[2].plot(solver.val_acc_history, label=update_rule)

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  adam
(Iteration 1 / 200) loss: 2.510546
(Epoch 0 / 5) train acc: 0.132000; val_acc: 0.136000
(Iteration 11 / 200) loss: 2.195202
(Iteration 21 / 200) loss: 2.078113
(Iteration 31 / 200) loss: 1.812428
(Epoch 1 / 5) train acc: 0.380000; val_acc: 0.325000
(Iteration 41 / 200) loss: 1.826273
(Iteration 51 / 200) loss: 1.863777
(Iteration 61 / 200) loss: 1.567749
(Iteration 71 / 200) loss: 1.867368
(Epoch 2 / 5) train acc: 0.425000; val_acc: 0.333000
(Iteration 81 / 200) loss: 1.527325
(Iteration 91 / 200) loss: 1.566109
(Iteration 101 / 200) loss: 1.519420
(Iteration 111 / 200) loss: 1.472078
(Epoch 3 / 5) train acc: 0.500000; val_acc: 0.357000
(Iteration 121 / 200) loss: 1.429839
(Iteration 131 / 200) loss: 1.318583
(Iteration 141 / 200) loss: 1.370015
(Iteration 151 / 200) loss: 1.398510
(Epoch 4 / 5) train acc: 0.547000; val_acc: 0.396000
(Iteration 161 / 200) loss: 1.324741
(Iteration 171 / 200) loss: 1.226352
(Iteration 181 / 200) loss: 1.351348
(Iteration 191 / 200) loss: 1.434819
(Epoch 5 / 5) train acc: 0.572000; val_acc: 0.372000

Running with  rmsprop
(Iteration 1 / 200) loss: 2.913589
(Epoch 0 / 5) train acc: 0.116000; val_acc: 0.129000
(Iteration 11 / 200) loss: 2.127827
(Iteration 21 / 200) loss: 1.958287
(Iteration 31 / 200) loss: 1.864374
(Epoch 1 / 5) train acc: 0.386000; val_acc: 0.307000
(Iteration 41 / 200) loss: 1.856374
(Iteration 51 / 200) loss: 1.676541
```

```
(Iteration 61 / 200) loss: 1.861262
(Iteration 71 / 200) loss: 1.756344
(Epoch 2 / 5) train acc: 0.434000; val_acc: 0.324000
(Iteration 81 / 200) loss: 1.678805
(Iteration 91 / 200) loss: 1.772977
(Iteration 101 / 200) loss: 1.515064
(Iteration 111 / 200) loss: 1.550025
(Epoch 3 / 5) train acc: 0.472000; val_acc: 0.345000
(Iteration 121 / 200) loss: 1.477977
(Iteration 131 / 200) loss: 1.625532
(Iteration 141 / 200) loss: 1.551768
(Iteration 151 / 200) loss: 1.424575
(Epoch 4 / 5) train acc: 0.511000; val_acc: 0.348000
(Iteration 161 / 200) loss: 1.572805
(Iteration 171 / 200) loss: 1.398190
(Iteration 181 / 200) loss: 1.502438
(Iteration 191 / 200) loss: 1.393758
(Epoch 5 / 5) train acc: 0.526000; val_acc: 0.364000
```




**Inline Question 2**

AdaGrad uses the update rule

```
cache += dw ** 2
w += -learning_rate * dw / (np.sqrt(cache) + eps)
```

Why can AdaGrad's updates become very small over time? Would Adam have the same issue?

**Answer.**

AdaGrad keeps adding squared gradients into `cache`, so the denominator grows monotonically. Even if the current gradient stays informative, the effective step size keeps shrinking because it is divided by a larger and larger accumulated history. That can make learning stall.

Adam is much less prone to that exact failure mode because it uses **exponentially decaying moving averages** of both the gradient and the squared gradient. In other words, Adam forgets old information instead of accumulating it forever, so the denominator does not grow without bound in the same way.

### 5.10 Train a strong fully connected model

Now train the best fully connected model you can on CIFAR-10 and store it in `best_model`. A reasonable target is **at least 50% validation accuracy**; with careful tuning, higher results are possible.

This repo version keeps the search intentionally small so it stays within roughly **5 minutes** on a typical local setup. Expand the grid later if you want a stronger model.

[12]:
```
best_model = None
best_val_acc = -1.0
best_stats = None

configs = [
    {
        'hidden_dims': [256, 256],
        'learning_rate': 1e-3,
        'weight_scale': 2e-2,
        'reg': 1e-3,
        'normalization': 'batchnorm',
        'update_rule': 'adam',
        'num_epochs': 10,
        'batch_size': 200,
    }
]

for cfg in configs:
    model = FullyConnectedNet(
        cfg['hidden_dims'],
        reg=cfg['reg'],
        weight_scale=cfg['weight_scale'],
        normalization=cfg['normalization'],
    )

    solver = Solver(
        model,
        data,
        update_rule=cfg['update_rule'],
        optim_config={'learning_rate': cfg['learning_rate']},
        lr_decay=0.95,
        num_epochs=cfg['num_epochs'],
        batch_size=cfg['batch_size'],
        print_every=100,
```

```
        verbose=True,
    )

    solver.train()

    train_acc = solver.train_acc_history[-1]
    val_acc = solver.best_val_acc

    print(
        f"hidden_dims={cfg['hidden_dims']}, "
        f"lr={cfg['learning_rate']}, "
        f"weight_scale={cfg['weight_scale']}, "
        f"reg={cfg['reg']}, "
        f"normalization={cfg['normalization']}, "
        f"update_rule={cfg['update_rule']}, "
        f"train_acc={train_acc:.4f}, val_acc={val_acc:.4f}"
    )

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model = model
        best_stats = {
            **cfg,
            'train_acc': train_acc,
            'val_acc': val_acc,
        }

print('best_val_acc:', best_val_acc)
print('best_stats:', best_stats)
```

```
(Iteration 1 / 2450) loss: 2.524807
(Epoch 0 / 10) train acc: 0.186000; val_acc: 0.209000
(Iteration 101 / 2450) loss: 1.739812
(Iteration 201 / 2450) loss: 1.539538
(Epoch 1 / 10) train acc: 0.473000; val_acc: 0.477000
(Iteration 301 / 2450) loss: 1.544700
(Iteration 401 / 2450) loss: 1.471033
(Epoch 2 / 10) train acc: 0.543000; val_acc: 0.488000
(Iteration 501 / 2450) loss: 1.385581
(Iteration 601 / 2450) loss: 1.277765
(Iteration 701 / 2450) loss: 1.356322
(Epoch 3 / 10) train acc: 0.563000; val_acc: 0.493000
(Iteration 801 / 2450) loss: 1.322443
(Iteration 901 / 2450) loss: 1.264477
(Epoch 4 / 10) train acc: 0.594000; val_acc: 0.532000
(Iteration 1001 / 2450) loss: 1.241587
(Iteration 1101 / 2450) loss: 1.440785
(Iteration 1201 / 2450) loss: 1.118807
```

```
(Epoch 5 / 10) train acc: 0.612000; val_acc: 0.547000
(Iteration 1301 / 2450) loss: 1.211359
(Iteration 1401 / 2450) loss: 1.217150
(Epoch 6 / 10) train acc: 0.626000; val_acc: 0.546000
(Iteration 1501 / 2450) loss: 1.242872
(Iteration 1601 / 2450) loss: 1.173966
(Iteration 1701 / 2450) loss: 1.251989
(Epoch 7 / 10) train acc: 0.644000; val_acc: 0.554000
(Iteration 1801 / 2450) loss: 1.288293
(Iteration 1901 / 2450) loss: 1.213800
(Epoch 8 / 10) train acc: 0.678000; val_acc: 0.562000
(Iteration 2001 / 2450) loss: 1.126817
(Iteration 2101 / 2450) loss: 1.152198
(Iteration 2201 / 2450) loss: 1.215220
(Epoch 9 / 10) train acc: 0.654000; val_acc: 0.565000
(Iteration 2301 / 2450) loss: 1.135547
(Iteration 2401 / 2450) loss: 1.157126
(Epoch 10 / 10) train acc: 0.716000; val_acc: 0.560000
hidden_dims=[256, 256], lr=0.001, weight_scale=0.02, reg=0.001,
normalization=batchnorm, update_rule=adam, train_acc=0.7160, val_acc=0.5650
best_val_acc: 0.565
best_stats: {'hidden_dims': [256, 256], 'learning_rate': 0.001, 'weight_scale':
0.02, 'reg': 0.001, 'normalization': 'batchnorm', 'update_rule': 'adam',
'num_epochs': 10, 'batch_size': 200, 'train_acc': np.float64(0.716), 'val_acc':
np.float64(0.565)}
```

### 5.11 Final evaluation

Run the best model on the validation and test sets. As in the earlier notebooks, saving the final model makes it easy to reuse later without retraining.

[13]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)

print('Validation set accuracy: ', np.mean(y_val_pred == data['y_val']))
print('Test set accuracy: ', np.mean(y_test_pred == data['y_test']))
```

```
Validation set accuracy:  0.55
Test set accuracy:  0.552
```

[14]:
```
# Save best model
best_model.save('best_fully_connected_net.npy')
```

```
best_fully_connected_net.npy saved.
```

### 5.12 Takeaways

This notebook extends the earlier two-layer network exercise in two important ways:

- **depth** gives the model more expressive power, but also makes optimization more fragile;

- **optimizer choice** matters much more once the network gets deeper;
- **small-set overfitting tests** are one of the fastest ways to debug an implementation;
- **gradient checks** are still the most reliable correctness check before large training runs.

That combination of modular implementation and careful optimization is the main lesson of the fully connected network assignment.